

Paterni ponašanja u sistemu CampusEats

1. Uvod

CampusEats predstavlja informacioni sistem namijenjen upravljanju procesima rezervacije obroka, pregledom menija, dostavom, upravljanjem zaliha i generisanjem QR kodova. Sistem koriste različite korisničke uloge kao što su studenti, administratori, kuhinjsko osoblje i dostavljači.

U okviru dosadašnjeg razvoja sistema primijenjeni su određeni obrasci dizajna, uključujući Facade i Decorator pattern. Iako oni pojednostavljaju pojedine dijelove sistema, javlja se potreba za dodatnim unapređenjem komunikacije između komponenti koje učestvuju u procesu rezervacije i dostave obroka.

2. Analiza postojećeg sistema

Na osnovu klasnog dijagrama može se uočiti da centralnu ulogu imaju klase Korisnik, Rezervacija, Dostava, QRKod, Obrok i Zaliha. Kada korisnik izvrši rezervaciju obroka, kroz sistem prolazi više koraka koji uključuju provjeru dostupnosti obroka, potvrdu rezervacije, generisanje QR koda i eventualno kreiranje dostave.

Promjena statusa rezervacije utiče na više učesnika sistema. Korisnik treba biti informisan o promjeni statusa, kuhinja mora znati kada započeti pripremu obroka, a dostavljač kada je obrok spreman za preuzimanje. U postojećoj realizaciji ove komponente postaju međusobno zavisne, što otežava održavanje i proširenje sistema.

3. Identifikacija problema

Glavni problem predstavlja potreba za obavještanjem više različitih komponenti nakon promjene stanja jednog objekta. Ukoliko bi svaka klasa direktno pozivala ostale klase, došlo bi do velikog broja međusobnih zavisnosti.

Takav pristup otežava dodavanje novih funkcionalnosti. Na primjer, ukoliko bi se u budućnosti željelo dodati slanje email obavještenja ili mobilnih notifikacija, bilo bi potrebno mijenjati postojeće klase, što povećava složenost sistema i mogućnost nastanka grešaka.

4. Predloženi patern ponašanja – Observer

Kao rješenje predlaže se uvođenje Observer paterna. Observer omogućava da jedan objekat automatski obavijesti sve zainteresovane objekte kada dođe do promjene njegovog stanja.

U CampusEats sistemu klase Rezervacija i Dostava mogu imati ulogu subjekta (Subject), dok Korisnik, Admin, Kuhinja i Dostavljač mogu imati ulogu posmatrača (Observer). Kada se promijeni status rezervacije ili dostave, svi registrovani posmatrači automatski dobijaju obavještenje o nastaloj promjeni.

5. Primjena Observer paterna u CampusEats sistemu

Prilikom kreiranja rezervacije korisnik postaje zainteresovana strana za sve naredne promjene njenog statusa. Kada kuhinja potvrdi rezervaciju, korisnik automatski dobija informaciju da je rezervacija prihvaćena. Nakon završetka pripreme obroka dostavljač dobija obavještenje da može preuzeti narudžbu.

Na isti način administrator može pratiti sve važne događaje u sistemu bez potrebe za dodatnim provjerama baze podataka. Observer pattern omogućava centralizovano i automatsko obavješćavanje svih učesnika procesa.

6. Prednosti uvođenja Observer paterna

Primjenom Observer paterna postiže se manja povezanost između klasa, bolja organizacija programskog koda i jednostavnije održavanje sistema. Dodavanje novih vrsta obavještenja moguće je bez izmjene postojećih poslovnih pravila.

Pored toga, sistem postaje fleksibilniji i spremniji za buduća proširenja. Uvođenje novih korisničkih uloga ili novih načina obavješćavanja ne zahtijeva značajne izmjene postojećih komponenti.

7. Veza sa postojećim obrascima dizajna

Observer pattern se prirodno nadovezuje na već implementirane obrasce. Facade pattern i dalje upravlja složenim procesom rezervacije, dok Decorator omogućava proširenje funkcionalnosti obroka. Observer dopunjava postojeću arhitekturu tako što rješava problem komunikacije između komponenti sistema.

8. Zaključak

Analizom postojećeg CampusEats sistema ustanovljena je potreba za uvođenjem dodatnog paterna ponašanja radi efikasnijeg upravljanja promjenama statusa rezervacija i dostava. Kao najpogodnije rješenje predložen je Observer pattern.

Njegovom primjenom omogućava se automatsko obavješćavanje korisnika, kuhinje, dostavljača i administratora, uz istovremeno smanjenje međusobne zavisnosti klasa. Time sistem postaje pregledniji, fleksibilniji i pogodniji za dalji razvoj.